

1. Load and Preprocess Data

- Read CSV file without headers and assign column names `ca` and `oa`.

	ca	oa
0	1279	3533
1	1289	3543
2	1296	3568
3	1289	3554
4	1285	3532
...	...	...
11995	1277	3518
11996	1296	3567
11997	1292	3550
11998	1274	3536
11999	1277	3522

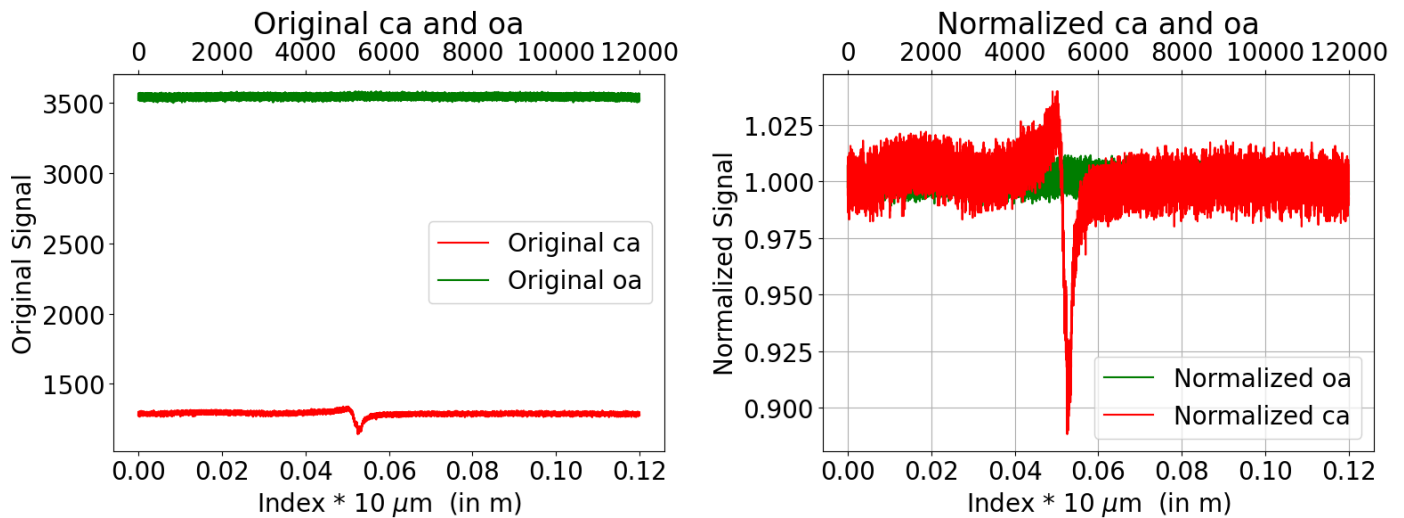
12000 rows × 2 columns

- Normalization of the original signal:
 - Calculate the mean of the first and last 500 rows of each signal.
 - Normalize signals `ca` and `oa` using these means.
 - Filter out rows where normalized values are zero (if any).

	ca	oa	normalized_ca	normalized_oa
0	1279	3533	0.994056	0.997552
1	1289	3543	1.001828	1.000376
2	1296	3568	1.007268	1.007434
3	1289	3554	1.001828	1.003481
4	1285	3532	0.998719	0.997270
...	...	...	...	...
11995	1277	3518	0.992501	0.993317
11996	1296	3567	1.007268	1.007152
11997	1292	3550	1.004159	1.002352
11998	1274	3536	0.990170	0.998399
11999	1277	3522	0.992501	0.994446

12000 rows × 4 columns

2. Visualize Original and Normalized Data



3. Identify Region of Interest

- **Select a subset** of normalized data between index 4000 and 7000.
- **Find positions** of maximum and minimum values in this range.
- **Extract data** between these extrema and create a temporary DataFrame with:
 - `'x'`: index values
 - `'normalized_ca'`: corresponding signal values

	x	normalized_ca
0	4905	1.039911
1	4906	1.032139
2	4907	1.022035
3	4908	1.019704
4	4909	1.023590
...	...	...
345	5250	0.897681
346	5251	0.929547
347	5252	0.905453
348	5253	0.915557
349	5254	0.888355

4. Linear Fit to Extract Transformation Parameters

- **Define cropping bounds:**
 - Determine the region of interest in the data.
 - Add a 10% padding around this region to capture the relevant neighborhood.
- **Extract cropped data:**
 - From the full dataset, extract:
 - `mid_data_y` = normalized transmittance (independent variable)
 - `mid_data_x` = corresponding index values (dependent variable).

350 rows × 2 columns

- **Perform linear regression:**
 - Fit a straight line of the form:

$$x = m*y + b;$$

Where y = normalized transmittance,
x = index value,

m = slope of the regression
line, and b = intercept.

- **Predict x when y = 1:**

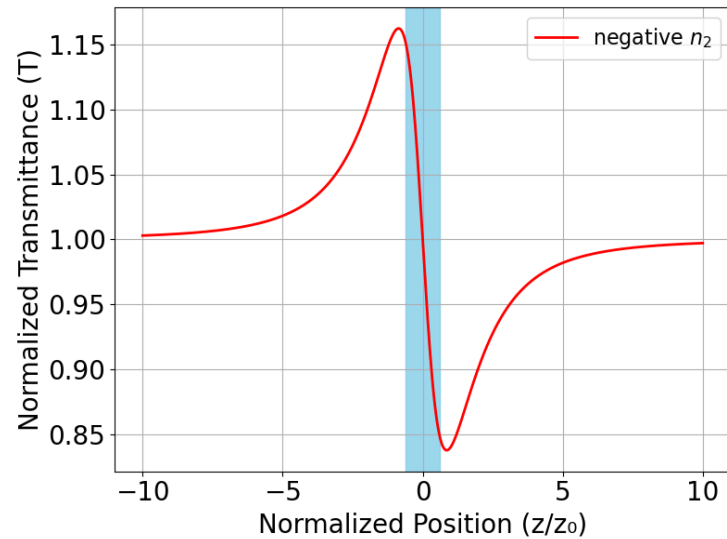
- Substitute y=1 into the fitted
line equation:

$$x_{\text{predict}} = m*(y=1) + b$$

Or substitute y=1 into the
fitted line equation [see
[appendix. Figure \(a, b\)](#)]:

$$y = m * x_{\text{predict}} + b$$

$$\text{or, } x_{\text{predict}} = y*(1/m) - (b/m)$$



5. Coordinate Transformation

- **Define a** transformation function to
compute the new **x_position**:

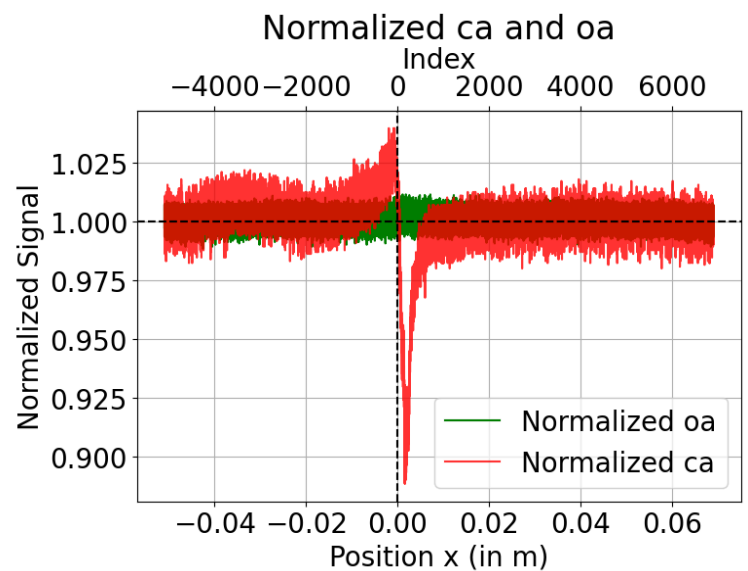
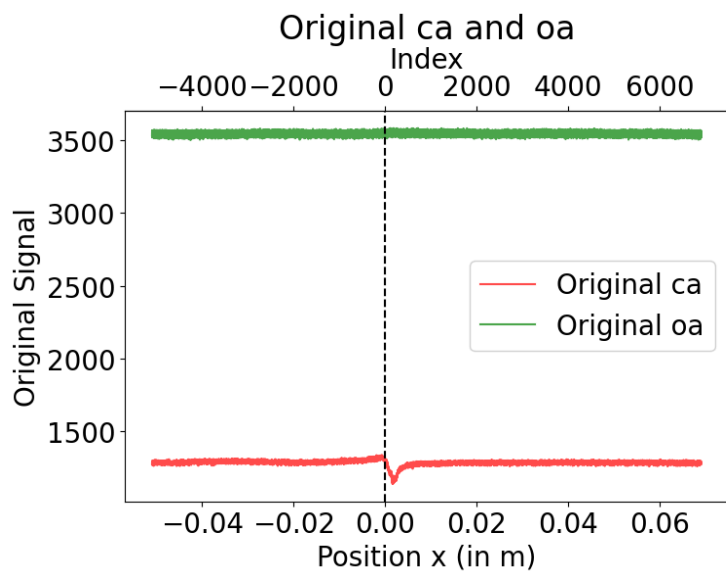
$$x_{\text{position}} = (\text{index} - x_{\text{pred}}) * 1e-5$$

- **Apply this** transformation and add
x_position as a new column in
the DataFrame.

	ca	oa	normalized_ca	normalized_oa	x_position
0	1279	3533	0.994056	0.997552	-0.050907
1	1289	3543	1.001828	1.000376	-0.050897
2	1296	3568	1.007268	1.007434	-0.050887
3	1289	3554	1.001828	1.003481	-0.050877
4	1285	3532	0.998719	0.997270	-0.050867
...	...	...	...	...	...
11995	1277	3518	0.992501	0.993317	0.069043
11996	1296	3567	1.007268	1.007152	0.069053
11997	1292	3550	1.004159	1.002352	0.069063
11998	1274	3536	0.990170	0.998399	0.069073
11999	1277	3522	0.992501	0.994446	0.069083

12000 rows x 5 columns

6. Re-Visualize with Transformed Coordinates



Appendix:

Figure (a): $x_{\text{predict}} = \text{slope} * y(=1) + \text{intercept}$

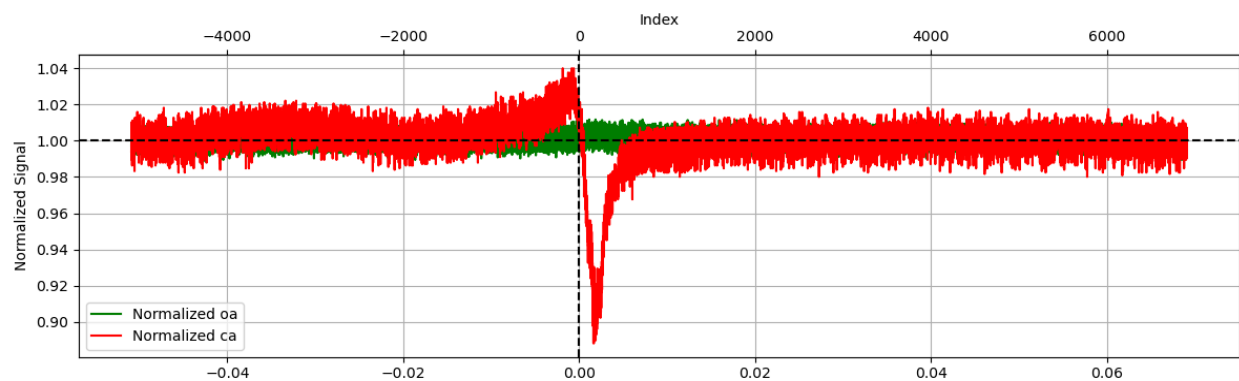
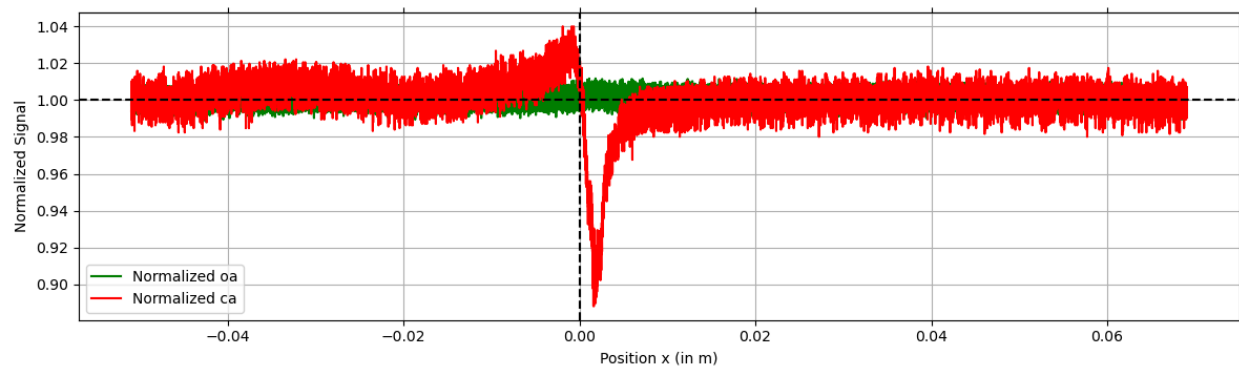


Figure (b): $x_{\text{predict}} = (1/\text{slope}) * y(=1) - (\text{intercept}/\text{slope})$

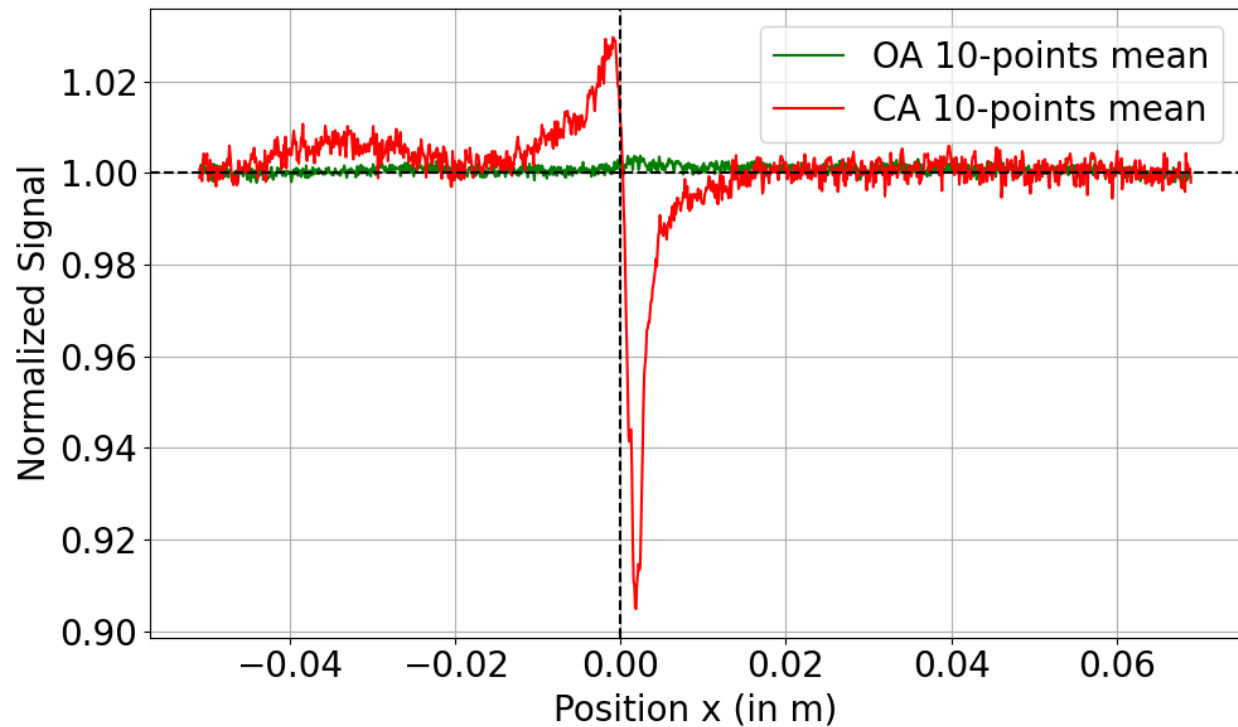


7. Reduce Data Points Using Block-Wise Aggregation

- Define a function `reduce_data` to:
 - Group data into chunks (e.g., every 10 rows).
 - Compute the mean and standard deviation of selected columns within each group.
- Apply this to get `reduced_df` with lower resolution but statistical summaries.

	x_position	normalized_ca_mean	normalized_ca_std	normalized_oa_mean	normalized_oa_std
0	-0.050907	0.998952	0.007589	1.000771	0.004885
1	-0.050807	1.001361	0.006992	1.000884	0.004685
2	-0.050707	0.999962	0.004709	1.001816	0.004722
3	-0.050607	0.998253	0.006954	1.000573	0.004624
4	-0.050507	1.002683	0.006340	0.999698	0.005027
...	...	...	...	...	...
1195	0.068593	1.002139	0.006597	0.999387	0.005023
1196	0.068693	1.000973	0.007504	0.999218	0.004944
1197	0.068793	1.000740	0.006133	1.000884	0.004791
1198	0.068893	1.000973	0.006233	0.999077	0.005825
1199	0.068993	0.997942	0.007437	0.999020	0.005196

1200 rows × 5 columns



8. Filter Region of Interest

- Further filter the reduced data to a specific range (e.g., $-0.03 < x < 0.03$).

	x_position	normalized_ca_mean	normalized_ca_std	normalized_oa_mean	normalized_oa_std
210	-0.029907	1.010222	0.006644	1.001900	0.004834
211	-0.029807	1.007657	0.005511	1.000460	0.004128
212	-0.029707	1.004004	0.006161	0.999980	0.004615
213	-0.029607	1.006569	0.006403	0.999585	0.004933
214	-0.029507	1.008512	0.006708	1.001364	0.004923
...	...	...	...	...	...
805	0.029593	1.001439	0.005703	1.000319	0.004935
806	0.029693	0.999574	0.004597	1.000686	0.004471
807	0.029793	0.996232	0.005883	1.000150	0.004992
808	0.029893	1.000506	0.008002	1.000488	0.005336
809	0.029993	0.999030	0.005412	1.000376	0.004462

600 rows × 5 columns

- Plot the mean signals (`normalized_ca`, `normalized_oa`) within this window.



In case of Nonlinear Absorption Present

